LocalScript: локальная агентская система для генерации Lua-кода

РТУ МИРЭА
Самодуров Никита

НИУ ВШЭ
Бочарский Григорий

РТУ МИРЭА
Иванов Кирилл

Оглавление

1 Проблематика

2 Концепция решения

3 Архитектура

4 Оркестрация и жизненный цикл запроса

5 Clarifier (Агент уточнения)

6 Namer (Названия чатов)

7 Generator и База Знаний

8 Validator (Агент валидации)

9 Работа с данными и историей

10 Инфраструктура LLM

11 Технологический стек

12 Перспектива развития

13 Наша команда

Проблематика

1

Почему обычные подходы не работают

Безопасность

Облачные LLM нельзя использовать в энтерпрайзе

Утечка NDA и корпоративных данных

Изоляция

Работа в закрытых контурах без интернета

API внешних провайдеров недоступны

Качество

Маленькие локальные модели часто ошибаются

Синтаксические баги в генерируемом коде

Как сделать AI-помощника по коду внутри изолированного контура?

Без облака · без утечек · без потери качества

Концепция решения

2

Мультиагентная система компенсирует слабость одной модели

Обычный подход

Одна LLM

Маленькая локальная модель

- ✗ Синтаксические ошибки
- ✗ Логические баги без проверки
- ✗ Нет уточнения требований

Подход LocalScript

Clarifier → **Generator** → **Validator**

Та же модель, разделение ролей и самопроверка

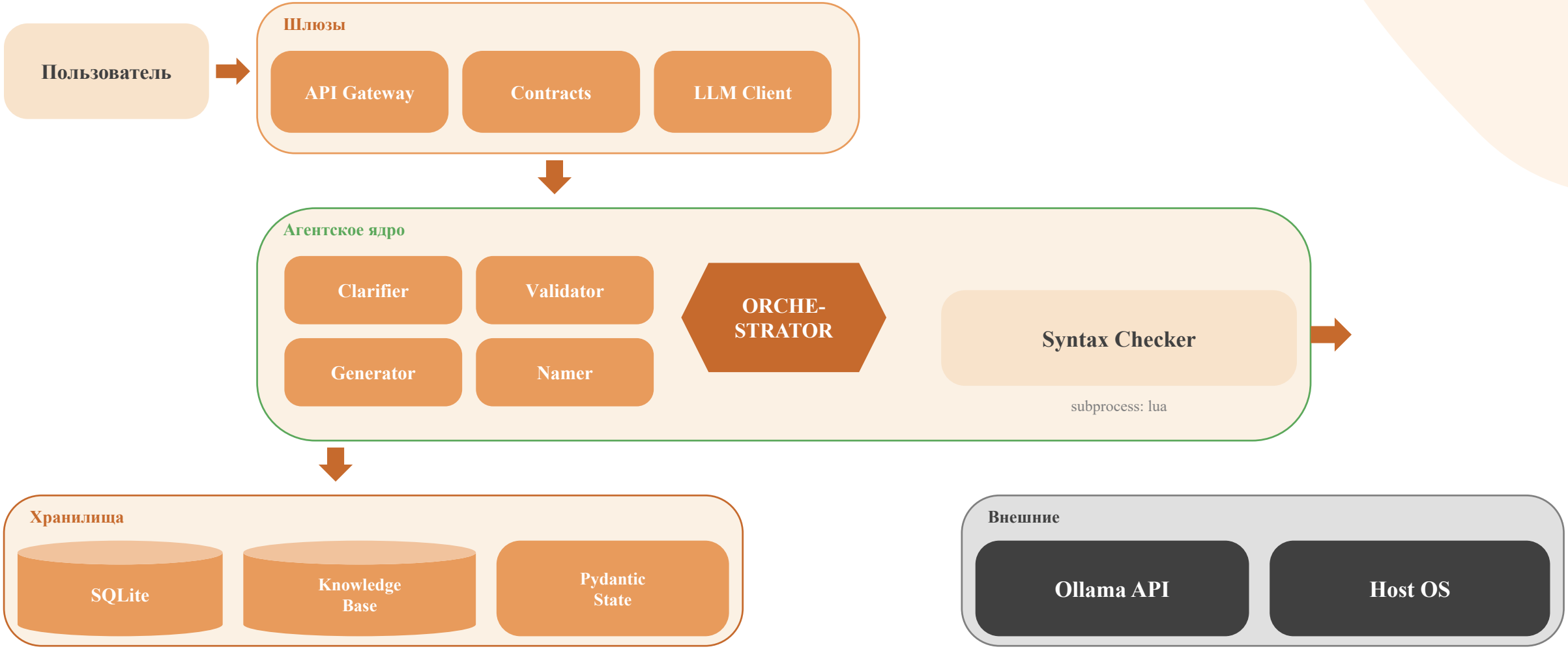
- ✓ Уточнение до генерации
- ✓ Few-shot из Knowledge Base
- ✓ Двойная валидация + retry

100% On-Premise · никаких внешних API · работа в закрытом контуре

Архитектура

3

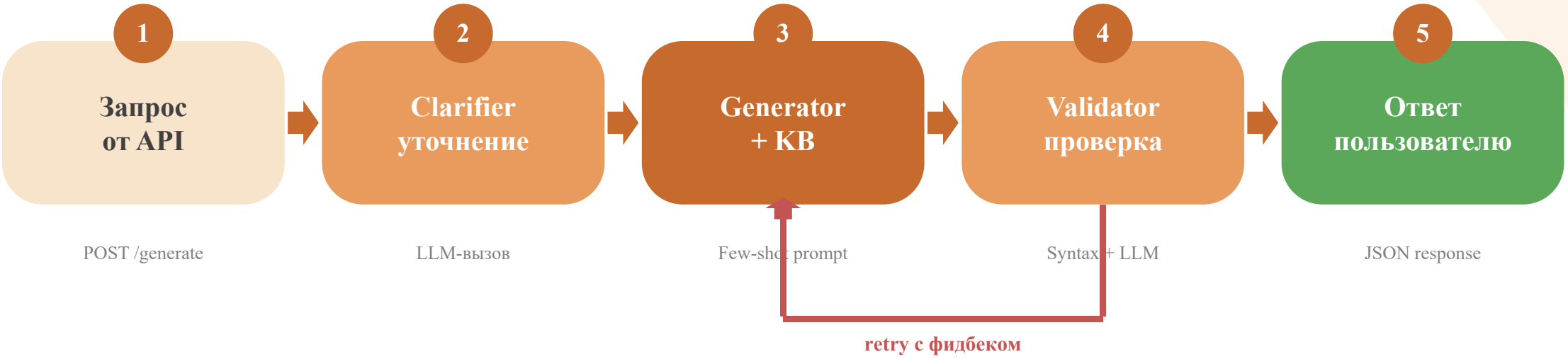
Изолированный бэкенд из трёх слоёв



Оркестрация

4

Жизненный цикл запроса в



Что делает Orchestrator

1. **Граф состояний**
2. **Маршрутизация** — вызов нужного агента
3. **Retry-логика** — при ошибке валидации
4. **Логирование** — в SQLite session

при первом промпте в сессии дополнительно вызывается Namer для генерации названия сессии

Claryfier

Зачем нужен

— Преобразует абстрактный запрос в чёткое ТЗ
— Уточняет неоднозначности до генерации
— Снижает количество retry-итераций

Как работает

— Получает сырой промпт через Orchestrator
— Делает LLM-вызов через Unified Client
— Возвращает структурированный Task

«Сделай скрипт логирования»

Сырой промпт от пользователя

Clarifier Agent

LLM-вызов: уточняющие вопросы

Task(
цель, входы, формат)

Структурированное ТЗ → Generator

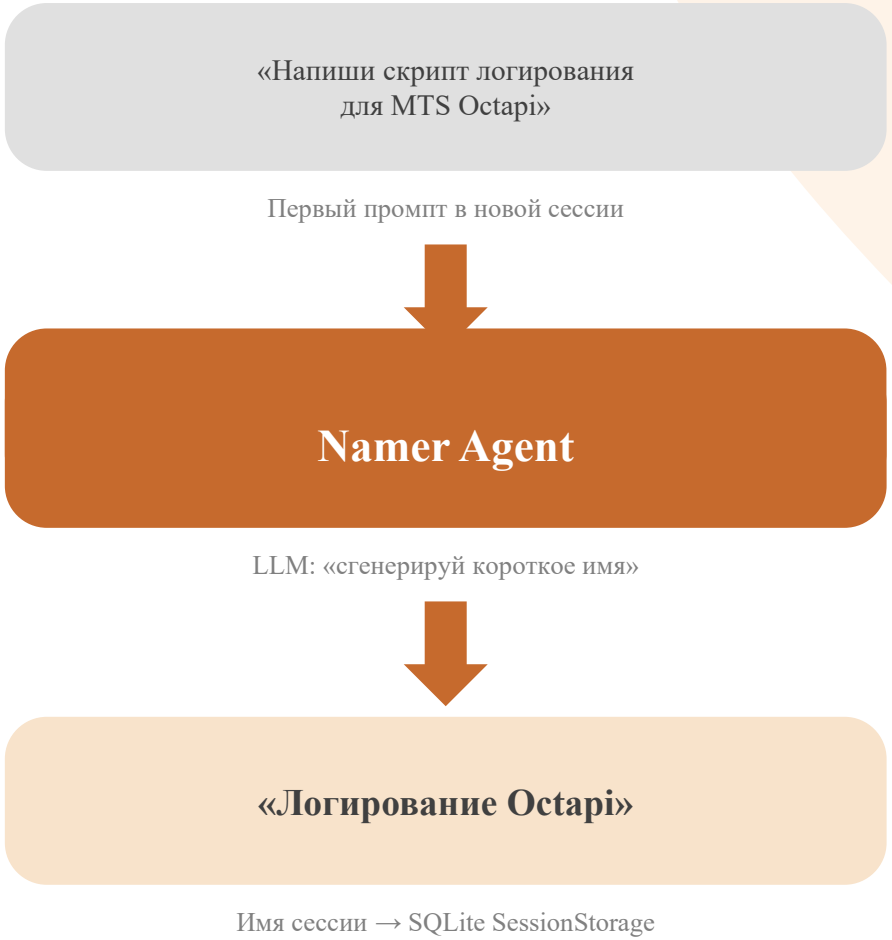
Namer

Зачем нужен

— Генерирует название сессии по первому промпту
— Делает историю чатов читаемой в UI
— Срабатывает один раз на старте сессии

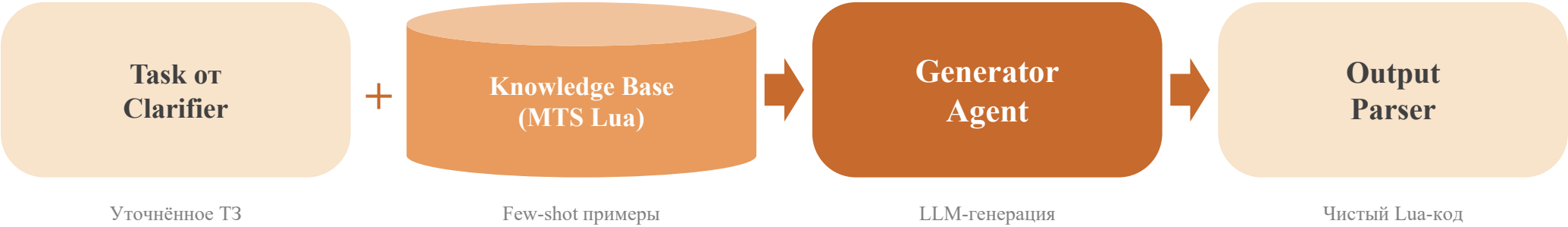
Как работает

— Получает первый user-промпт от Orchestrator
— Делает короткий LLM-вызов через Unified Client
— Возвращает строку-название и сохраняет в SQLite



Generator

Синтез Lua-кода с использованием эталонных примеров



Что делает Generator

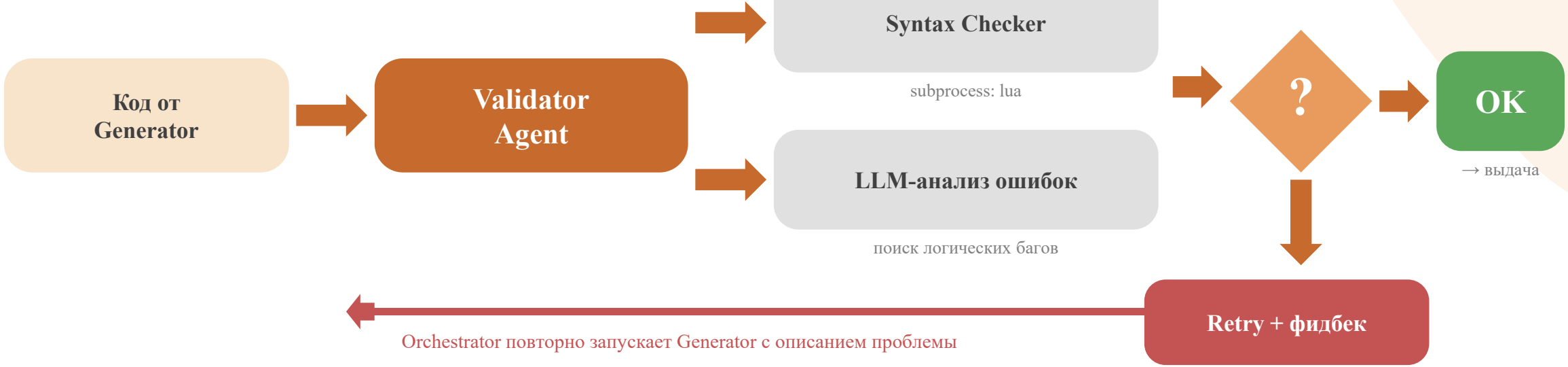
- Подмешивает примеры из Knowledge Base
- Few-shot prompting повышает качество на маленьких моделях

Что делает Output Parser

- Извлекает Lua-код регэкспом по код-блокам
- Возвращает чистый код в Orchestrator

Validator

Двухуровневая проверка с retry-циклом



Лимит итераций предотвращает бесконечный цикл

Validator проверяет код двумя независимыми способами — формальным и семантическим — и решает, готов ли результат к выдаче

Данные и история

Хранение состояния и контекста между агентами

SQLite

SessionStorage

- История чатов + имя от Namer
- Восстановление контекста
- Без внешних зависимостей

ORCHESTRATOR

Pydantic Models

Стейт между агентами

- Контракты Task и CodeResult
- In-memory передача
- Типобезопасность на стыках

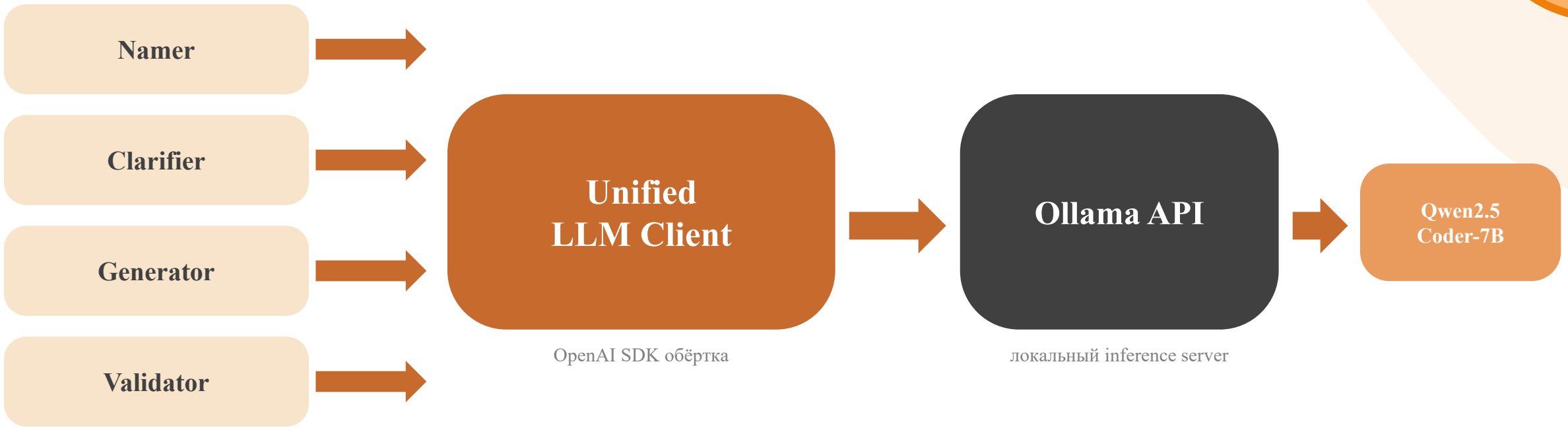
KB

Static Knowledge Base

Статичный набор эталонных примеров MTS Lua — основа для few-shot prompting у Generator

Инфраструктура LLM

Единая точка LLM-инференса для всех агентов



100% offline · одна точка для смены модели · единый API для всех агентов

Замена модели не требует правок в коде агентов — только конфиг Ollama

Технологический стек

Backend	Python 3.12	FastAPI API Gateway	Pydantic контракты	
LLM и инфра	Ollama inference	Qwen2.5-Coder 7B	OpenAI SDK клиент	
Хранилища	SQLite session	lua subprocess	luac syntax check	Docker деплой

Перспектива развития

Roadmap развития проекта

Now

MVP

- Базовая ReAct-петля
- Static Knowledge Base
- SQLite session

Q3 2026

Расширение

- RAG поверх MTS docs
- Поддержка Python/Go
- Web UI

Q4 2026

Production

- Docker on-premise
- Prometheus метрики
- Multi-tenant

2027

Tuning

- Fine-tune на корп. коде
- LoRA-адаптеры
- Кастом эмбеддинги

Наша команда

Самодуров Никита

ML-engineer

РТУ МИРЭА

@Professional_228

Бочарский Григорий

Backend Developer

НИУ ВШЭ

@gangsize_von

Иванов Кирилл

ML-engineer

РТУ МИРЭА

@K1r1ll1010